Ingeniería del Software II

Tema 7

Verificación Y Pruebas del Sistema

- **Tema 7. Verificación Y Pruebas del Sistema.** Verificación y validación. Planificación de la verificación y validación. Inspecciones de software. Análisis estático automatizado. Verificación y métodos formales. Pruebas del software: Pruebas del sistema. Pruebas de componentes. Diseño de casos de prueba. Pruebas Orientadas a Objetos**.** Automatización de las pruebas. Validación de sistemas críticos.

Verificación Y Pruebas del Sistema

Las pruebas sólo son una de las técnicas de verificación y validación. Algunas de estas técnicas, tales como las inspecciones de programas, han sido utilizadas durante casi treinta años , pero todavía no se han convertido en tendencias principales de la ingeniería del software.

Deberán tratar a lo largo de los siguientes temas, las aproximaciones para verificar que el software satisface su especificación y validar que también satisface las necesidades del cliente del software.

Verificación Y Pruebas del Sistema

- Nunca se dará suficiente importancia a las pruebas del software y sus implicaciones en la calidad del software. Citando a Deutsch [DEU79]:-

Verificación Y Pruebas del Sistema

El proceso de pruebas del software tiene dos objetivos distintos:

1. Para demostrar al desarrollador y al cliente que el software satisface sus requerimientos. Para el software a medida, esto significa que debería haber al menos una prueba para cada requerimiento de los documentos de requerimientos del sistema y del usuario. Para productos de software genéricos, significa que debería haber pruebas para todas las características del sistema que se incorporarán en la entrega del producto.
2. Para descubrir defectos en el software en que el comportamiento de éste es incorrecto, no deseable o no cumple su especificación. La prueba de defectos está relacionada con la eliminación de todos los tipos de comportamientos del sistema no deseables, tales como caídas del sistema, interacciones no permitidas con otros sistemas, cálculos incorrectos y corrupción de datos.

Verificación y validación

Durante y después del proceso de implementación, el programa que se está desarrollando debe ser comprobado para asegurar que satisface su especificación y entrega la funcionalidad esperada por las personas que pagan por el software. ***La verificación y la validación (V & V)*** *es* el nombre dado a estos procesos de análisis y pruebas. La verificación y la validación tienen lugar en cada etapa del proceso del software. V & V comienza con revisiones de los requerimientos y continúa con revisiones del diseño e inspecciones de código hasta la prueba del producto.

Verificación y validación

La verificación y la validación no son lo mismo, aunque a menudo se confunden. Boehm (Boehm, 1979) expresó de forma sucinta la diferencia entre ellas:

- «Validación: ¿Estamos construyendo el producto correcto?»
- «Verificación: ¿Estamos construyendo el producto correctamente?»

Verificación y validación

La verificación implica comprobar que el software está de acuerdo con su especificación. Debería comprobarse que satisface sus requerimientos funcionales y no funcionales.

La validación, es un proceso más general. El objetivo de la validación es asegurar que el sistema software satisface las expectativas del cliente. Va más allá de la comprobación de que el sistema satisface su especificación para demostrar que el software hace lo que el cliente espera que haga.

Verificación y validación

El objetivo último del proceso de verificación y validación es establecer la seguridad de que el sistema software está «hecho para un propósito». Esto significa que el sistema debe ser lo suficientemente bueno para su uso pretendido.

El nivel de confianza requerido depende del propósito del sistema, las expectativas de los usuarios del sistema y el entorno de mercado actual del sistema:

Verificación y validación

1. ***Función del software.*** El nivel de confianza requerido depende de lo crítico que sea el software para una organización. Por ejemplo, el nivel de confianza requerido para el software que se utiliza para controlar un sistema de seguridad crítico es mucho más alto que el requerido para un prototipo de un sistema software que ha sido desarrollado para demostrar algunas ideas nuevas.

Verificación y validación

2 ***Expectativas del usuario.*** Una reflexión lamentable sobre la industria del software es que muchos usuarios tienen pocas expectativas sobre su software y no se sorprenden cuando éste falla durante su uso. Están dispuestos a aceptar estos fallos del sistema cuando los beneficios de su uso son mayores que sus desventajas. Sin embargo, la tolerancia de los usuarios a los fallos de los sistemas está decreciendo desde los años 90. Actualmente es menos aceptable entregar sistemas no fiables, por lo que las compañías de software deben invertir más esfuerzo para verificar y validar.

Verificación y validación

3. ***Entorno de mercado.*** Cuando un sistema se comercializa, los vendedores del sistema deben tener en cuenta los programas competidores, el precio que sus clientes están dispuestos a pagar por el sistema y la agenda requerida para entregar dicho sistema. Cuando una compañía tiene pocos competidores, puede decidir entregar un programa antes de que haya sido completamente probado y depurado, debido a que quiere ser el primero en el mercado. Cuando los clientes no están dispuestos a pagar precios altos por el software, pueden estar dispuestos a tolerar más defectos en él. Todos estos factores pueden considerarse cuando se decide cuánto esfuerzo debería invertirse en el proceso de V & V.

Verificación y validación

Dentro del proceso de V & V, existen dos aproximaciones complementarias para el análisis y comprobación de los sistemas:

1. ***Las inspecciones de software*** analizan y comprueban las representaciones del sistema tales como el documento de requerimientos, los diagramas de diseño y el código fuente del programa. Puede usarse las inspecciones en todas las etapas del proceso. Las inspecciones pueden ser complementadas con algún tipo de análisis automático del código fuente de un sistema o de los documentos asociados. Las inspecciones de software y los análisis automáticos son técnicas de V & V estáticas, ya que no se necesita ejecutar el software en una computadora.

Verificación y validación

2. ***Las pruebas del software*** **i**mplican ejecutar una implementación del software con datos de prueba. Se examinan las salidas del software y su entorno operacional para comprobar que funciona tal y como se requiere. Las pruebas son una técnica dinámica de verificación y validación.

Verificación y validación

Sólo puede probarse un sistema cuando está disponible un prototipo o una versión ejecutable del programa. Una ventaja del desarrollo incremental es que una versión probable del sistema está disponible en etapas tempranas del proceso de desarrollo. Las funcionalidades pueden probarse a medida que se van añadiendo al sistema, por lo que no tiene que realizarse una implementación completa antes de que comiencen las pruebas.

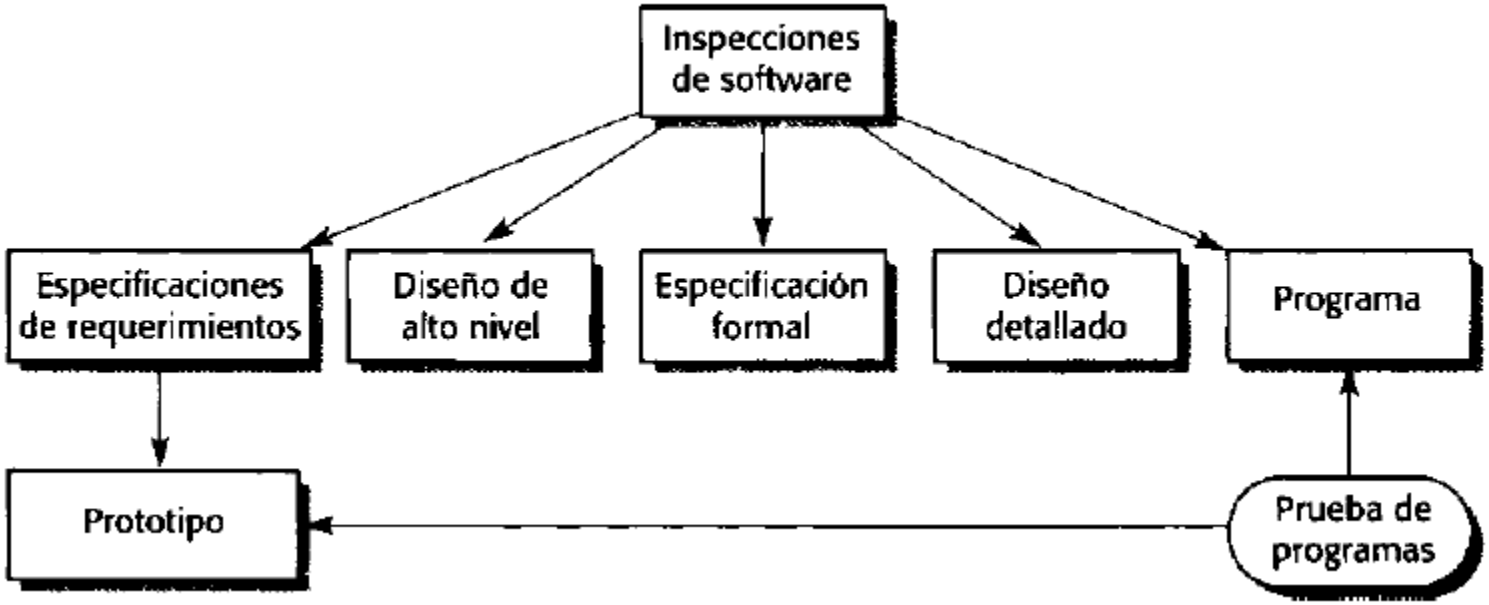
Verificación y validación

Las técnicas de inspección comprenden las inspecciones de programas, el análisis automático del código fuente y la verificación formal. Sin embargo, las técnicas estáticas sólo pueden comprobar la correspondencia entre un programa y su especificación (verificación); no pueden demostrar que el software es operacionalmente útil. Tampoco se pueden utilizar técnicas estáticas para comprobar las propiedades emergentes del software tales como su rendimiento y fiabilidad.

Verificación y validación

Aunque el uso de las inspecciones del software no es generalizado, la prueba de programas siempre será la principal técnica de verificación y validación. Las pruebas implican ejecutar el programa utilizando datos similares a los datos reales procesados por el programa. Los defectos en los programas se descubren examinando las salidas del programa y buscando las anomalías

Verificación y validación



Verificación y validación

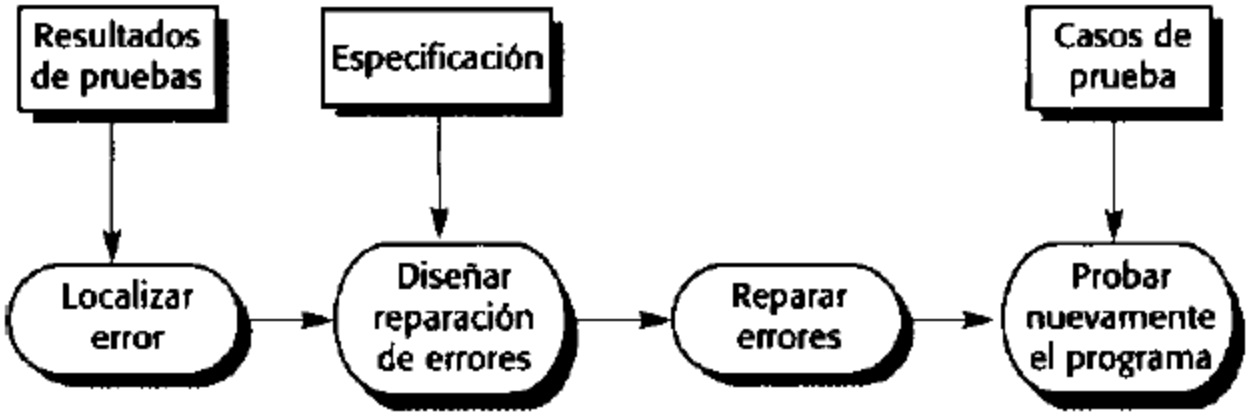
Existen dos tipos distintos de pruebas que pueden utilizarse en diferentes etapas del proceso del software:

1. ***Las pruebas de validación intentan demostrar que el software es el que el cliente quiere*** —que satisface sus requerimientos—. Como parte de la prueba de validación, se pueden utilizar pruebas estadísticas para probar el rendimiento y la fiabilidad de los programas, y para comprobar cómo trabaja en ciertas condiciones operacionales.
2. ***Las pruebas de defectos intentan revelar defectos en el sistema*** *en lugar de simular su* uso operacional. El objetivo de las pruebas de defectos es hallar inconsistencias entre un programa y su especificación.

Verificación y validación

Las pruebas (o más generalmente la verificación y validación) y la depuración tienen diferentes objetivos:

1. Los procesos de verificación y validación intentan establecer la existencia de defectos en el sistema software.
2. La depuración es un proceso que localiza y corrige estos defectos.



Verificación y validación

Las herramientas de depuración interactivas generalmente forman parte de un conjunto de herramientas de soporte del lenguaje que se integran con un sistema de compilación. Éstas proporcionan un entorno de ejecución especializado para el programa que permite acceder a la tabla de símbolos del compilador y, desde aquí, a los valores de las variables del programa. Se puede controlar la ejecución «paso a paso» del programa sentencia por sentencia. Después de ejecutar cada sentencia, pueden examinar los valores de las variables y así se puede descubrir la localización del defecto.

Verificación y validación

Cuando se ha descubierto un defecto en el programa, hay que corregirlo y volver a validar el sistema. Esto puede implicar volver a inspeccionar el programa o hacer pruebas de regresión en las que se ejecutan de nuevo los tests existentes. Las pruebas de regresión se utilizan para comprobar que los cambios en el programa no introducen nuevos defectos. La experiencia ha demostrado que una alta proporción de «reparaciones» de defectos son incompletas o bien introducen nuevos defectos en el programa.

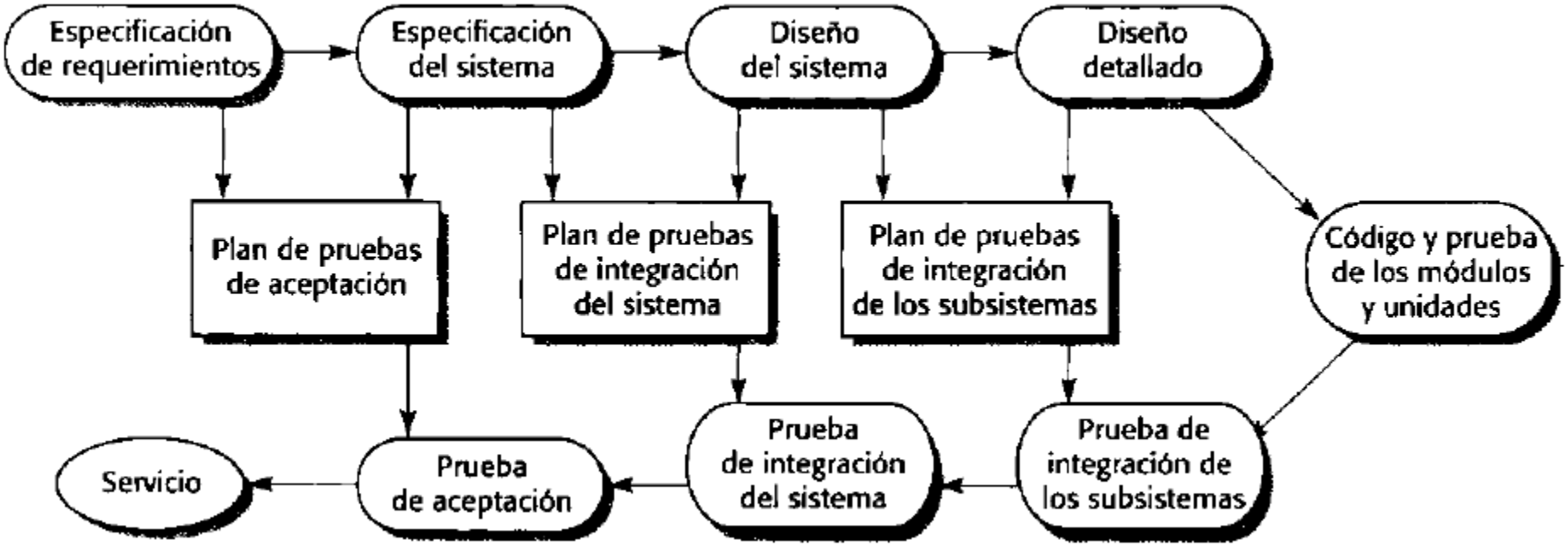
Debería poderse establecer una traza entre los casos de prueba y los componentes que son probados. Si esta trazabilidad se documenta, entonces se puede ejecutar un subconjunto de los casos de prueba del sistema para comprobar el componente modificado y sus dependientes.

Planificación de la verificación y validación

La verificación y validación es un proceso costoso. Para algunos sistemas, tales como los sistemas de tiempo real con restricciones no funcionales complejas, más de la mitad del presupuesto para el desarrollo del sistema puede invertirse en V & V. Es necesaria una planificación cuidadosa para obtener el máximo provecho de las inspecciones y pruebas y controlar los costos del proceso de verificación y validación.

Debería comenzarse la planificación de la verificación y validación del sistema en etapas tempranas del proceso de desarrollo.

Planificación de la verificación y validación



Planificación de la verificación y validación

La planificación de las pruebas está relacionada con el establecimiento de estándares para el proceso de las pruebas, no sólo con la descripción de los productos de las pruebas. Los planes de pruebas, además de ayudar a los gestores a asignar recursos y estimar el calendario de las pruebas, son de utilidad para los ingenieros del software implicados en el diseño y la realización de las pruebas del sistema. Estos ayudan al personal técnico a obtener una panorámica general de las pruebas del sistema y ubicar su propio trabajo en este contexto. Una buena descripción de los planes de pruebas y su relación con los planes de calidad más generales se proporciona en Frewin y Hatton (Frewin y Hatton, 1986). Humphrey (Humphrey, 1989) y Kit (Kit, 1995) también incluyen estudios sobre la planificación de las pruebas.

Planificación de la verificación y validación

Existen diferentes componentes para un plan de pruebas para un sistema grande y complejo. Además de determinar el calendario y procedimientos de las pruebas, el plan de pruebas define los recursos hardware y software que se requieren. Éste es útil para los gestores del sistema que son los responsables de asegurar que estos recursos están disponibles para el equipo de pruebas.

Planificación de la verificación y validación

Los planes de pruebas normalmente deberían incluir cantidades significativas de contingencias para que los desajustes en la implementación y el diseño puedan solucionarse y el personal pueda ser reasignado a otras actividades.

Para sistemas más pequeños, se puede utilizar un plan de pruebas menos formal, pero sigue siendo necesario un documento formal para soportar la planificación del proceso de pruebas.

Planificación de la verificación y validación

Para algunos procesos ágiles como la programación extrema, las pruebas son inseparables del desarrollo. Al igual que otras actividades de planificación, la planificación de las pruebas también es incremental. En XP, el cliente es el último responsable de decidir cuánto esfuerzo debería dedicarse a la prueba del sistema.

Los planes de pruebas no son documentos estáticos, sino que evolucionan durante el proceso de desarrollo. Los planes de pruebas cambian debido a retrasos en otras etapas del proceso de desarrollo. Si parte de un sistema está incompleto, el sistema no puede probarse como un todo. Entonces tiene que revisarse el plan de pruebas para volver a desplegar y a asignar a los encargados de las pruebas a alguna otra actividad, y recuperarlos cuando el software vuelva a estar disponible.

- El proceso de prueba.
- Descripción de las principales fases del proceso de prueba

Planificación de la verificación y validación

Los Requerimientos.

Los usuarios son los más interesados en que el sistema satisfaga sus requerimientos y las pruebas deberían planificarse para que todos los requerimientos se prueben individualmente.

Elementos probados

Deberían especificarse los elementos del proceso del software que tienen que ser probados.

Calendario de pruebas

Un calendario de todas las pruebas y la asignación de recursos para este calendario se enlaza, obviamente, con la agenda general del desarrollo del proyecto.

Planificación de la verificación y validación

Procedimientos de registro de las pruebas

No es suficiente ejecutar simplemente las pruebas; los resultados de las pruebas deben ser registrados sistemáticamente. Debe ser posible auditar el proceso de pruebas para comprobar que se ha llevado a cabo correctamente.

Requerimientos hardware y software

Esta sección debería determinar las herramientas software requeridas y la utilización estimada del hardware

Restricciones

En esta sección deberían anticiparse las restricciones que afectan al proceso de pruebas como la escasez de personal.

Inspecciones de software

Las inspecciones de software son un proceso de V & V estático en el que un sistema software se revisa para encontrar errores, omisiones y anomalías. Generalmente, las inspecciones se centran en el código fuente, pero puede inspeccionarse cualquier representación legible del software como los requerimientos o un modelo de diseño. Cuando se inspecciona un sistema, se utiliza conocimiento del sistema, su dominio de aplicación y el lenguaje de programación o modelo de diseño para descubrir errores.

Inspecciones de software

Existen tres ventajas fundamentales de la inspección sobre las pruebas:

1. Durante las pruebas, los errores pueden enmascarar (ocultar) otros errores. Cuando se descubre un error, nunca se puede estar seguro de si otras anomalías de salida son debidas a un nuevo error o son efectos laterales del error original. Debido a que la inspección es un proceso estático, no hay que preocuparse de las interacciones entre errores. Por lo tanto, una única sesión de inspección puede descubrir muchos errores en un sistema.

Inspecciones de software

2. Pueden inspeccionarse versiones incompletas de un sistema sin costos adicionales. Si un programa está incompleto, entonces se necesita desarrollar software de soporte especializado para las pruebas a fin de probar aquellas partes que están disponibles. Esto, obviamente, añade costos al desarrollo del sistema.

Inspecciones de software

3. Además de buscar los defectos en el programa, una inspección también puede considerar atributos de calidad más amplios de un programa tales como grado de cumplimiento con los estándares, portabilidad y mantenibilidad. Puede buscarse ineficiencias, algoritmos no adecuados y estilos de programación que podrían hacer que el sistema fuese difícil de mantener y actualizar.

Se han realizado varios estudios y experimentos que han demostrado que las inspecciones son más efectivas para descubrir defectos que las pruebas del programa.

Inspecciones de software

Las revisiones y las pruebas tienen cada una sus ventajas e inconvenientes y deberían utilizarse conjuntamente en el proceso de verificación y validación. En realidad, Gilb y Graham sugieren que uno de los usos más efectivos de las revisiones es la revisión de los casos de prueba para un sistema. Las revisiones pueden descubrir problemas con estos tests y ayudar a diseñar formas más efectivas para probar el sistema. Se puede empezar la V & V del sistema con inspecciones en etapas tempranas del proceso de desarrollo, pero una vez que se integra un sistema, se necesita comprobar sus propiedades emergentes y que la funcionalidad del sistema es la que su propietario realmente quiere.

Inspecciones de software

A pesar del éxito de las inspecciones, se ha demostrado que es difícil introducir las inspecciones formales en muchas organizaciones de desarrollo de software. Los ingenieros de software con experiencia en la prueba de programas a menudo son reacios a aceptar que las inspecciones pueden ser más efectivas para detectar defectos que las pruebas. Los gestores pueden ser reacios debido a que las inspecciones requieren costos adicionales durante el diseño y el desarrollo. Pueden no querer asumir el riesgo de que no obtendrán los correspondientes ahorros durante las pruebas de los programas.

No hay duda de que las inspecciones sobrecargan al inicio los costos de V & V del software y conducen a un ahorro de costos sólo después de que los equipos de desarrollo adquieran experiencia en su uso. Además, hay problemas prácticos en cuanto a la organización de las inspecciones: éstas requieren tiempo para organizarse y parecen ralentizar el proceso de desarrollo. Es difícil convencer a un gestor muy presionado que este tiempo puede recuperarse más tarde debido a que se tendrá que emplear menos tiempo depurando el programa.

El proceso de inspección de programas

Las inspecciones de programas son revisiones cuyo objetivo es la detección de defectos en el programa. El concepto de un proceso de inspección formalizado se desarrolló por primera vez por I BM en los años 70 (Fagan, 1976; Fagan, 1986). Actualmente es un método bastante utilizado de verificación de programas, especialmente en ingeniería de sistemas críticos. A partir del método original de Fagan, se han desarrollado varias aproximaciones alternativas de las inspecciones (Gilb y Graham, 1993). Todas ellas están basadas en un grupo con miembros que tienen diferentes conocimientos realizando una revisión cuidadosa línea por línea del código fuente del programa.

El proceso de inspección de programas

La diferencia principal entre las inspecciones de programas y otros tipos de revisiones de calidad es que el objetivo primordial de las inspecciones es encontrar defectos en el programa en lugar de considerar cuestiones de diseño más generales. Los defectos pueden ser errores lógicos, anomalías en el código que podrían indicar una condición errónea, o el incumplimiento de los estándares del proyecto o de la organización. Por otra parte, otros tipos de revisión pueden estar más relacionados con la agenda, los costos, el progreso frente a hitos definidos o la evaluación de si es probable que el software cumpla los objetivos fijados por la organización.

- La inspección de programas es un proceso formal realizado por un equipo de al menos cuatro personas. Los miembros del equipo analizan sistemáticamente el código y señalan posibles defectos. En las propuestas originales de Fagan, se sugieren roles tales como autor, lector, probador y moderador.

El proceso de inspección de programas

El lector lee el código en voz alta al equipo de inspección, el probador inspecciona el código desde una perspectiva de prueba y el moderador organiza el proceso.

A medida que las organizaciones ganan experiencia con la inspección, han surgido otras propuestas para los roles del equipo. En un estudio de cómo la inspección fue introducida con éxito en el proceso de desarrollo de Hewlett-Packard, Grady y Van Slack (Grady y Van Slack, 1994) sugieren seis roles. No creen que sea necesario leer el programa en voz alta. La misma persona puede adoptar más de un rol de forma que el tamaño del equipo puede variar de una inspección a otra. Gilb y Graham sugieren que los inspectores deberían ser seleccionados para reflejar diferentes puntos de vista tales como pruebas, usuario final y gestión de la calidad.

El proceso de inspección de programas

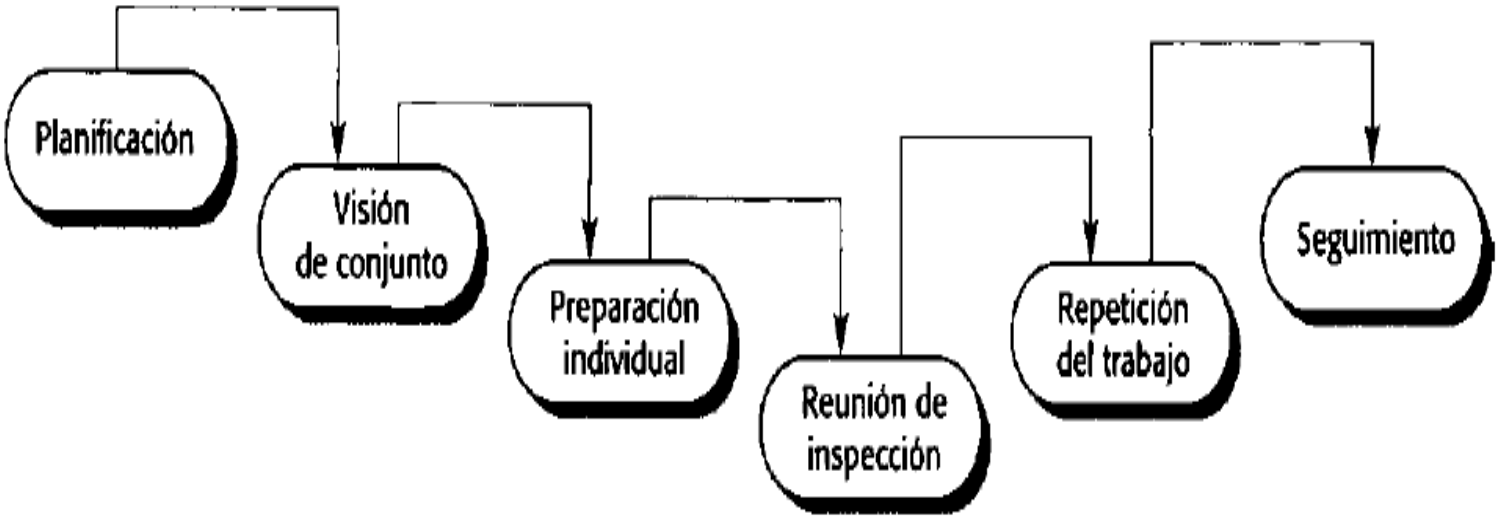
Autor o propietario	El programador o diseñador responsable de generar el programa o documento. Responsable de reparar los defectos descubiertos durante el proceso de inspección.
Inspector	Encuentra errores, omisiones e inconsistencias en los programas y documentos. También puede identificar cuestiones más generales que están fuera del ámbito del equipo de inspección.
Lector	Presenta el código o documento en una reunión de inspección.
Secretario	Registra los resultados de la reunión de inspección.
Presidente o moderador	Gestiona el proceso y facilita la inspección. Realiza un informe de los resultados del proceso para el moderador jefe.
Moderador jefe	Responsable de las mejoras del proceso de inspección, actualización de las listas de comprobación, estándares de desarrollo, etc.

El proceso de inspección de programas

Antes de que comience una inspección del programa, es esencial que:

1. Se tenga una especificación precisa del código a inspeccionar. Es imposible inspeccionar un componente a un nivel de detalle requerido para detectar defectos sin una especificación completa.
2. Los miembros del equipo de inspección estén familiarizados con los estándares de la organización.
3. Se haya distribuido una versión compilable y actualizada del código a todos los miembros del equipo. No existe ninguna razón para inspeccionar código que esté «casi completo » incluso si un retraso provoca desfases en la agenda.

El proceso de inspección de programas



El proceso de inspección de programas

La inspección en sí misma debería ser bastante corta (no más de dos horas) y debería centrarse en la detección de defectos, cumplimiento de los estándares y programación de baja calidad.

El equipo de inspección no debería sugerir cómo deben repararse estos defectos ni recomendar cambios en otros componentes.

A continuación de la inspección, el autor del programa debería realizar los cambios para corregir los problemas identificados. En la etapa siguiente, el moderador debería decidir si se requiere una reinspección de código. Puede decidir que no se requiere una reinspección completa y que los defectos han sido reparados con éxito. El programa entonces es aprobado por el moderador para su entrega.

Durante una inspección, a menudo se utiliza una lista de comprobación de errores de programación comunes para centrar el análisis. Esta lista de comprobación puede basarse en ejemplos de listas de comprobación de libros o en conocimiento de los defectos que son comunes en un dominio de aplicación particular. Se necesitan diferentes listas de comprobación para distintos lenguajes de programación debido a que cada lenguaje tiene sus propios errores característicos.

Análisis estático automatizado

Los analizadores estáticos son herramientas software que escanean el código fuente de un programa y detectan posibles defectos y anomalías. Analizan el código del programa y así reconocen los tipos de sentencias en el programa. Pueden detectar si las sentencias están bien formadas, hacer inferencias sobre el flujo de control del programa y, en muchos casos, calcular el conjunto de todos los posibles valores para los datos del programa. Complementan las facilidades de detección de errores proporcionadas por el compilador del lenguaje. Pueden utilizarse como parte del proceso de inspección o como una actividad separada del proceso V & V.

El objetivo del análisis estático automatizado es llamar la atención del inspector sobre las anomalías del programa, tales como variables que se utilizan sin inicialización, variables que no se usan o datos cuyo valor podría estar fuera de alcance.

Análisis estático automatizado

- Las etapas implicadas en el análisis estático comprenden:

1. ***Análisis del flujo de control.***.
2. ***Análisis del uso de los datos.***
3. ***Análisis de interfaces.***
4. ***Análisis del flujo de información.***
5. ***Análisis de caminos.***

Verificación y métodos formales

Los métodos formales de desarrollo del software se basan en representaciones matemáticas del software, normalmente como una especificación formal. Estos métodos formales se ocupan principalmente del análisis matemático de la especificación; de transformar la especificación a una representación mas detallada equivalente; o de verificar formalmente que una representación del sistema es semánticamente equivalente a otra representación.

Se podría pensar en el uso de métodos formales como la técnica de verificación estática. Los métodos formales requieren un análisis muy detallado de la especificación del sistema y del programa, y su uso consume a menudo tiempo y resulta caro. Como consecuencia, el uso de métodos formales esta restringido principalmente a los procesos de desarrollo de software de seguridad critico y seguro.

Pruebas del software.

- Las dos actividades fundamentales de pruebas son la prueba de componentes —probar las partes del sistema— y la prueba integral.

El proceso de pruebas del software tiene dos objetivos distintos:

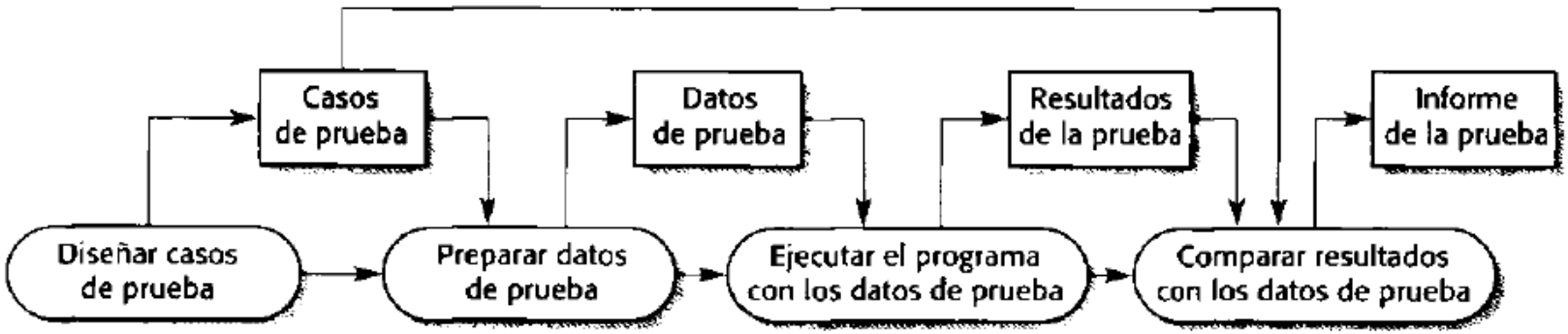
1. *Para demostrar al desarrollador y al cliente que el software satisface sus requerimientos.*
2. *Para descubrir defectos en el software en que el comportamiento de éste es incorrecto, no deseable o no cumple su especificación* del sistema —probar el sistema como un todo.

Pruebas del software:

- Las políticas de pruebas pueden basarse en la experiencia de uso del sistema y pueden centrarse en probar las características del sistema operacional. Por ejemplo:

1. Deberían probarse todas las funciones del sistema a las que se accede a través de menús.
2. Deben probarse todas las combinaciones de funciones (por ejemplo, formateado de textos) a las que se accede a través del mismo menú.
3. En los puntos del programa en los que el usuario introduce datos, todas las funciones deben probarse con datos correctos e incorrectos.

Pruebas del software:



- Modelo de Whittaker (Whittaker, 2002)

Pruebas del software:

Como una parte del proceso V & V, los gestores tienen que tomar decisiones sobre quién debería ser responsable de las diferentes etapas de las pruebas. Para la mayoría de los sistemas, los programadores tienen la responsabilidad de probar los componentes que ellos han desarrollado. Una vez que lo hacen, el trabajo se pasa a un equipo de integración que integra los módulos de diferentes desarrolladores, construye el software y prueba el sistema como un todo. Para sistemas críticos, puede utilizarse un proceso más formal en el que probadores independientes son responsables de todas las etapas del proceso de prueba.

Pruebas del software:

- Las pruebas del sistema, sin embargo, tienen que basarse en una especificación escrita del sistema.

Pruebas del sistema

Para la mayoría de los sistemas complejos, existen dos fases distintas de pruebas del sistema:

1. *Pruebas de integración,* en las que el equipo de pruebas tiene acceso al código fuente del sistema. Cuando se descubre un problema, el equipo de integración intenta encontrar la fuente del problema e identificar los componentes que tienen que ser depurados. Las pruebas de integración se ocupan principalmente de encontrar defectos en el sistema.

Pruebas del sistema

2. *Pruebas de entregas,* en las que se prueba una versión del sistema que podría ser entregada a los usuarios. Aquí, el equipo de pruebas se ocupa de validar que el sistema satisface sus requerimientos y con asegurar que el sistema es confiable. Las pruebas de entregas son normalmente pruebas de «caja negra» en las que el equipo de pruebas se ocupa simplemente de demostrar si el sistema funciona o no correctamente. Los problemas son comunicados al equipo de desarrollo cuyo trabajo es depurar el programa.

Pruebas del sistema

Cuando los clientes se implican en las pruebas de entregas, éstas a menudo se denominan *pruebas de aceptación*. *Si la entrega es lo suficientemente buena, el cliente* puede entonces aceptarla para su uso.

Pruebas de integración

La integración del sistema implica identificar grupos de componentes que proporcionan alguna funcionalidad del sistema e integrar éstos añadiendo código para hacer que funcionen conjuntamente. Algunas veces, primero se desarrolla el esqueleto del sistema en su totalidad, y se le añaden los componentes. Esto se denomina ***integración descendente.*** De forma alternativa, pueden integrarse primero los componentes de infraestructura que proporcionan servicios comunes, tales como el acceso a bases de datos y redes, y a continuación pueden añadirse los componentes funcionales. Ésta es la ***integración ascendente.***

En la práctica, para muchos sistemas, la estrategia de integración es una mezcla de ambas, añadiendo en incrementos componentes de infraestructura y componentes funcionales. En ambas aproximaciones de integración, normalmente tiene que desarrollarse código adicional para simular otros componentes y permitir que el sistema se ejecute.

Diseño de casos de prueba

El diseño de casos de prueba es una parte de las pruebas de componentes y sistemas en las que se diseñan los casos de prueba (entradas y salidas esperadas) para probar el sistema. El objetivo del proceso de diseño de casos de prueba es crear un conjunto de casos de prueba que sean efectivos descubriendo defectos en los programas y muestren que el sistema satisface sus requerimientos.

Para diseñar un caso de prueba, se selecciona una característica del sistema o componente que se está probando. A continuación, se selecciona un conjunto de entradas que ejecutan dicha característica, documenta las salidas esperadas o rangos de salida y, donde sea posible, se diseña una prueba automatizada que prueba que las salidas reales y esperadas son las mismas.

Diseño de casos de prueba

Existen varias aproximaciones que pueden seguirse para diseñar casos de prueba:

1. Pruebas basadas en requerimientos, en donde los casos de prueba se diseñan para probar los requerimientos del sistema. Esta aproximación se utiliza principalmente en la etapa de pruebas del sistema, ya que los requerimientos del sistema normalmente se implementan por varios componentes. Para cada requerimiento, se identifica casos de prueba que puedan demostrar que el sistema satisface ese requerimiento.

Diseño de casos de prueba

2. Pruebas estructurales, en donde se utiliza el conocimiento de la estructura del programa para diseñar pruebas que ejecuten todas las partes del programa. Esencialmente, cuando se prueba un programa, debería intentarse ejecutar cada sentencia al menos una vez. Las pruebas estructurales ayudan a identificar casos de prueba que pueden hacer esto posible.

Diseño de casos de prueba

3. Pruebas de particiones, en donde se identifican particiones de entrada y salida y se diseñan pruebas para que el sistema ejecute entradas de todas las particiones y genere salidas en todas las particiones. Las particiones son grupos de datos que tienen características comunes, como todos los números negativos, todos los nombres con menos de 30 caracteres, todos los eventos provocados por la elección de opciones en un menú, y así sucesivamente.

Automatización de las pruebas

- Un banco de pruebas del software es un conjunto integrado de herramientas para soportar el proceso de pruebas. Además de a los marcos de trabajo de pruebas que soportan la ejecución automática de las pruebas, un banco de trabajo puede incluir herramientas para simular otras partes del sistema y generar datos de prueba de dicho sistema.

Por lo tanto, los bancos de trabajo de pruebas completos, sólo se utilizan cuando se desarrollan sistemas grandes. Para estos sistemas, los costos totales de las pruebas pueden llegar al 50% del total de los costos de desarrollo, por lo que es rentable invertir en herramientas CASE de alta calidad para soportar las prueba

Muchas Gracias

- Consultas.
- La próxima clase…..

Bibliografía

- ‘Ingeniería de software un enfoque práctico’, Roger S. Pressman, 6th ed. Editorial MC Graw Hill. 2005, Capitulo 17.
- ‘Ingeniería del Software’, Ian Sommerville. 7th ed. Editorial Prentice Hall. Capitulo 22, 23, 24
- Apuntes Cátedra Análisis de Sistemas II, Facena UNNE. http://exa.unne.edu.ar/informatica/anasistem2/public_html/home.htm